

Nornir / NAPALM / Netmiko Demo — “Typist to Architect”

FOR
DUMMIES



Run these from a deployed lab. `./lab automation` (repo root) offers them as a menu and activates the venv for you; `./lab status` confirms the lab is up first.

This is the low-friction automation path: pure Python, no Ansible infrastructure required. It mirrors the Netmiko → Nornir → NAPALM progression from the team’s automation course, retargeted at the Containerlab cEOS topology instead of the original GNS3 + bastion-host lab.

Zero setup required to start. Containerlab bakes cEOS’s default `admin/admin` credentials directly into the auto-generated `nornir-simple-inventory.yml` it writes on every deploy. As long as you’ve run `../link-inventory.sh` once after deploying the lab, these scripts just work — no vault, no collections to install, no credentials to type in. That contrast is worth pointing out to coworkers evaluating whether to invest in full Ansible tooling: this is what “automation with (almost) zero prep work” looks like.

The venv these scripts run in is built with `uv` by `scripts/00-bootstrap.sh` (falling back to plain `pip` if `uv` isn’t available) — see the root README’s [Tooling choices](#) for why, and for the pure-`uv`-project alternative (`cd automation && uv sync`).

Run them in order

```
netauto # activate the venv (alias from ~/.bashrc)
cd automation/nornir
```

```
python3 01_netmiko_hello.py      # one device, raw Netmiko, "fire and forget"
python3 02_nornir_scale.py       # same idea, every device, concurrently
python3 03_napalm_facts.py       # structured JSON instead of screen-scraped text
python3 04_idempotent_deploy.py  # the payoff: declarative, idempotent config push
python3 04_idempotent_deploy.py  # run it again — nothing changes the 2nd time
```

Script	Concept	What to point out to coworkers
01_netmiko_hello.py	Imperative, single device	This is exactly what typing into an SSH session looks like as code — the easy on-ramp.
02_nornir_scale.py	Concurrency + inventory	Same script logic, but it now scales to however many devices are in the topology without a <code>for</code> loop.

Script	Concept	What to point out to coworkers
03_napalm_facts.py	Structured data	Compare the JSON output here to the raw text from script 1 — this is what makes step 4 possible.
04_idempotent_deploy.py	Declarative + idempotent	Run it twice. First run shows a diff and pushes it; second run shows changed: False. This is the actual “self-healing network” pitch.
05_spineleaf_underlay.py	Real per-host data at fabric scale	Same model as 04 with a real IP plan: deploys the full OSPF underlay to the 4-node spine-leaf topology. Used by course module 04; needs <code>topologies/spine-leaf.clab.yml</code> deployed and its inventory linked.

The teaching moment on secrets

Notice none of the scripts above needed a password typed in anywhere — that’s specific to this lab, because containerlab already knows cEOS’s default credentials. In a real network you’d never rely on default credentials being baked into an inventory file.

This repo sets up three different ways to override them, each a real Linux/DevOps tool worth knowing on its own merits, not just a “how to do secrets” lesson:

```
./run-with-env.sh    python3 02_nornir_scale.py    # A: plaintext .env, gitignored
./run-with-pass.sh  python3 02_nornir_scale.py    # B: pass (GPG-backed password store)
./run-with-sops.sh  python3 02_nornir_scale.py    # C: sops + age (encrypted-at-rest, git-committable)
```

All three just set `NORNIR_USERNAME` / `NORNIR_PASSWORD` in the environment before the script runs; `creds.py` (imported by every script above) picks them up if present and overrides the inventory defaults — otherwise it’s a no-op and you get the zero-setup behavior described above.

Full walkthrough of each, including install steps and when you’d actually reach for each one, in [SECRETS.md](#). It’s a good one to read side by side with the Ansible Vault setup in `../ansible/` — four approaches to the same problem, ranging from zero ceremony to production-grade, and worth showing coworkers as a “how much tooling do you actually need” comparison rather than a single “this is the answer” lesson.